

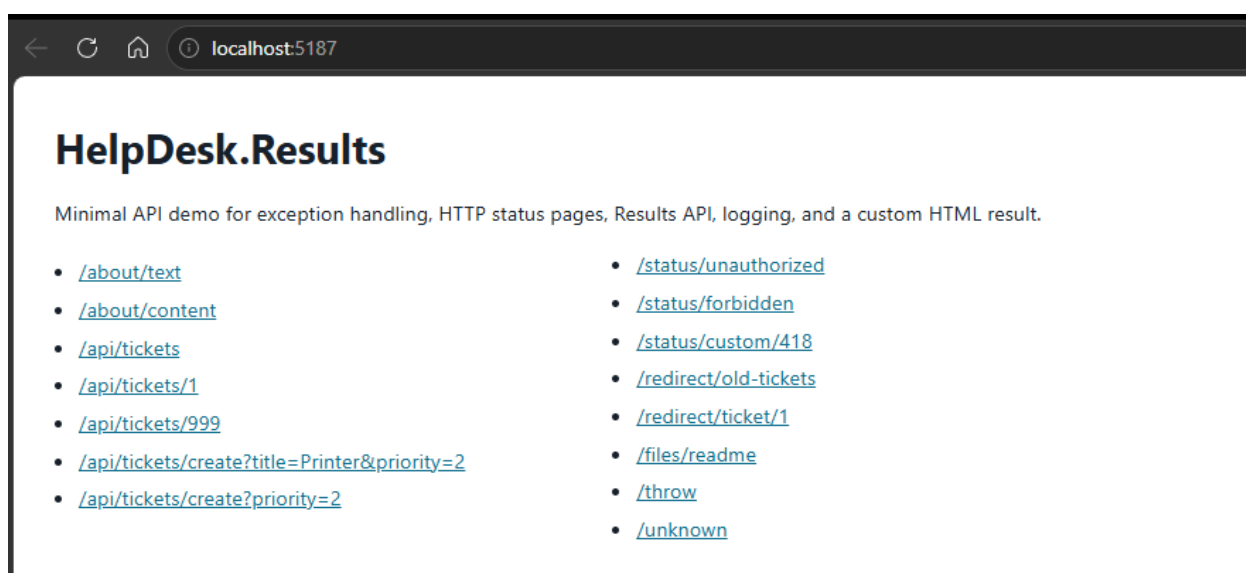
## Практическая работа 7-8

### 1. Цель работы

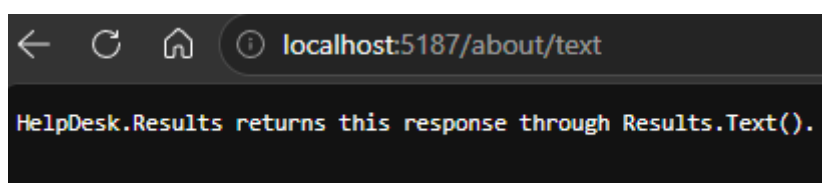
Научиться строить Minimal API, корректно обрабатывать исключения и штатные HTTP-ошибки, возвращать различные типы ответов через Results API, создавать собственный тип IActionResult и понимать влияние порядка middleware на формирование HTTP-ответа.

### 2. Демонстрация работы обязательных маршрутов

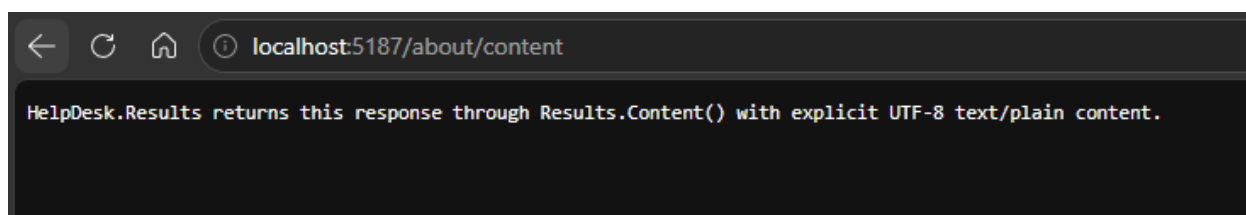
/



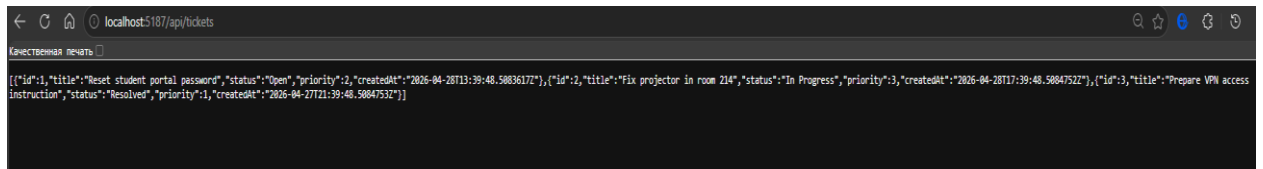
/about/text



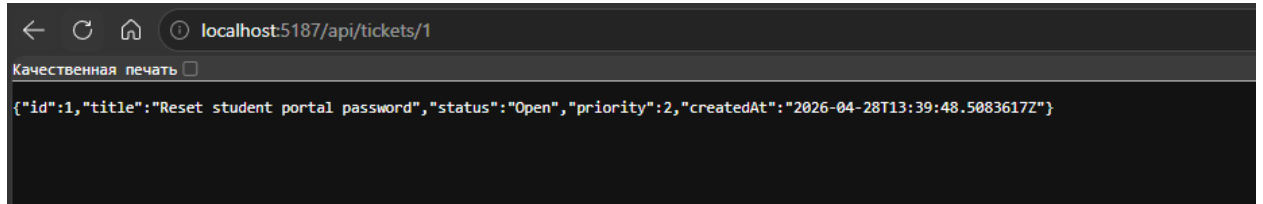
/about/content



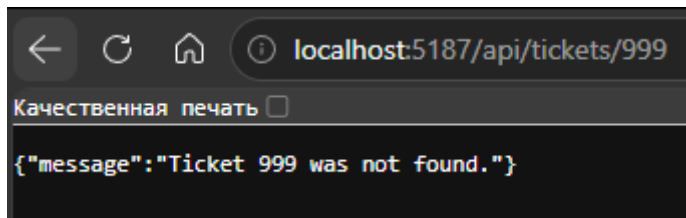
/api/tickets



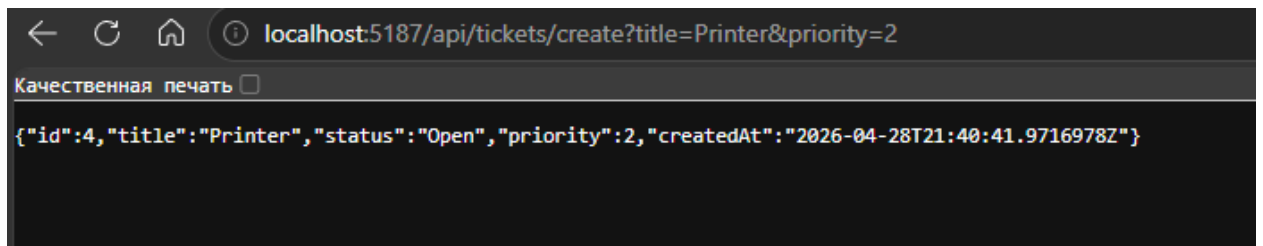
/api/tickets/1



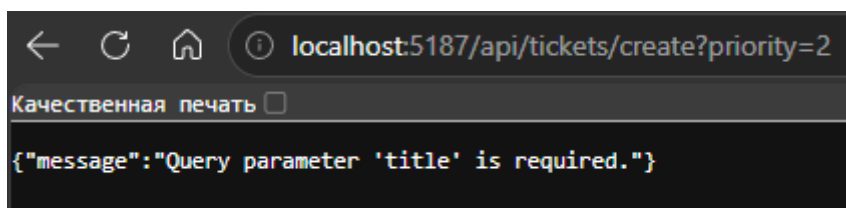
/api/tickets/999



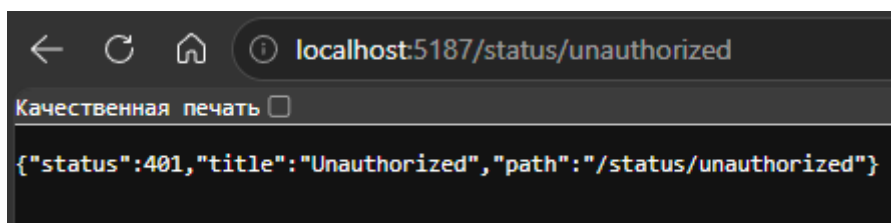
/api/tickets/create?title=Printer&priority=2



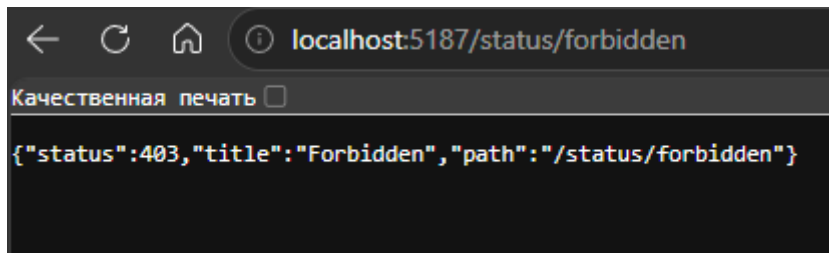
/api/tickets/create?priority=2



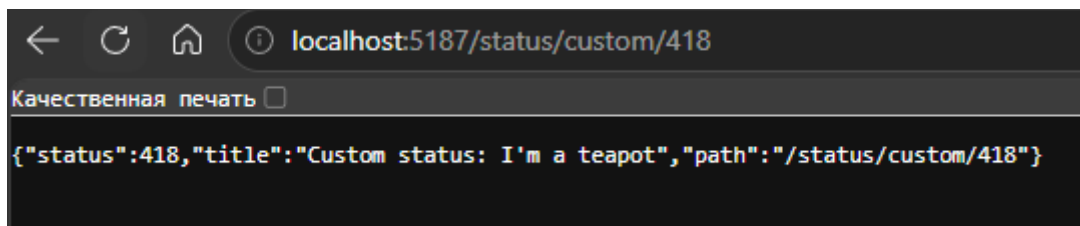
/status/unauthorized



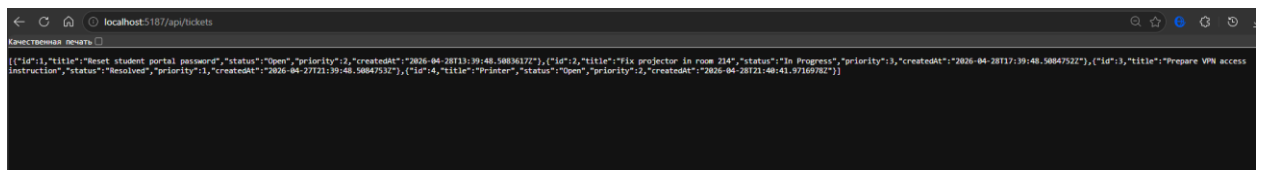
/status/forbidden



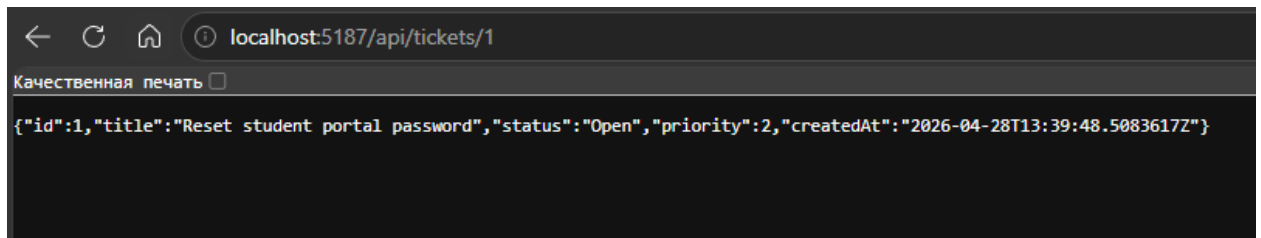
/status/custom/418



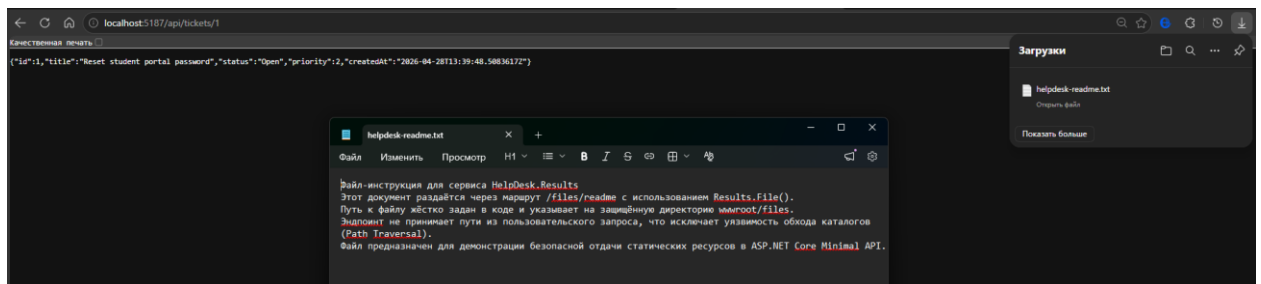
/redirect/old-tickets



/redirect/ticket/1



/files/readme

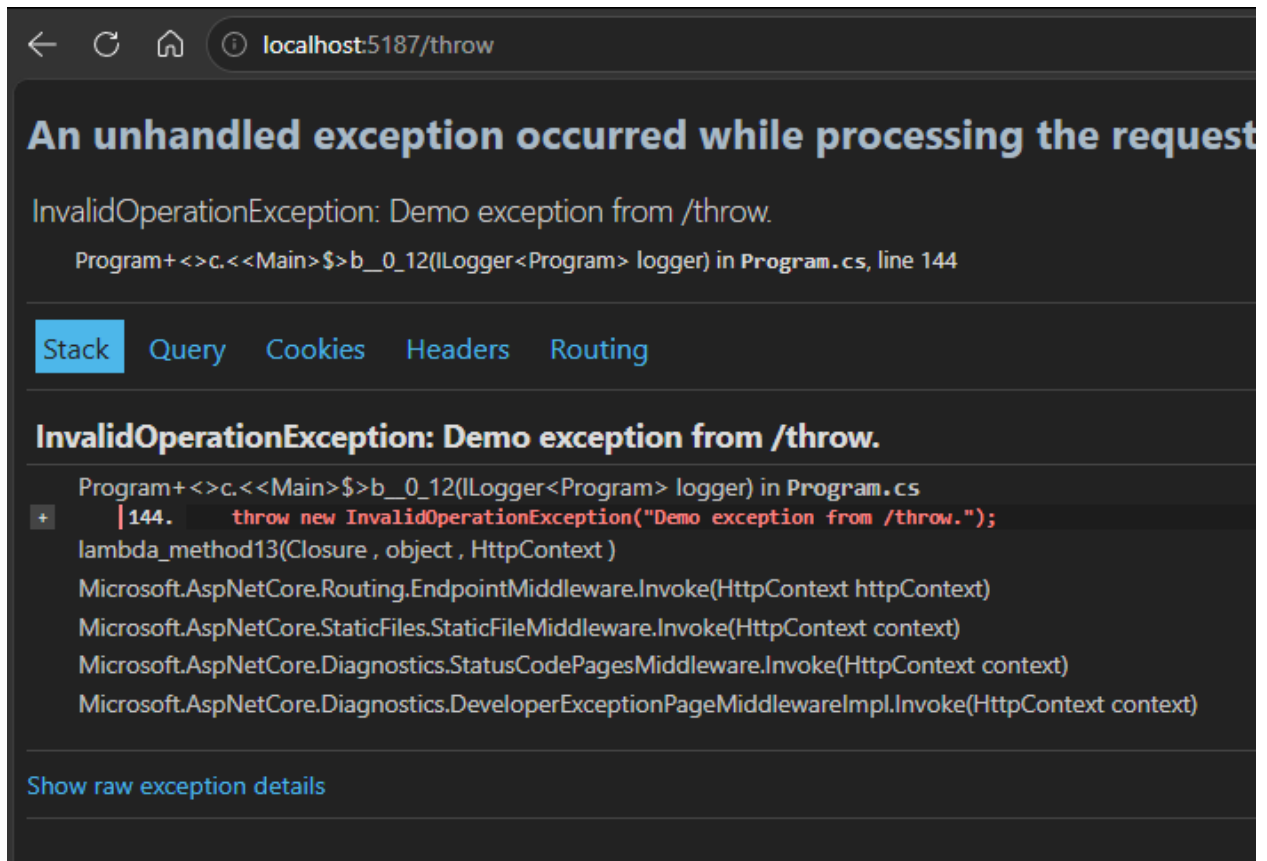


### 3. Обработка исключений (/throw)

#### 3.1. Среда Development

При `ASPNETCORE_ENVIRONMENT=Development` подключается `UseDeveloperExceptionPage()`.

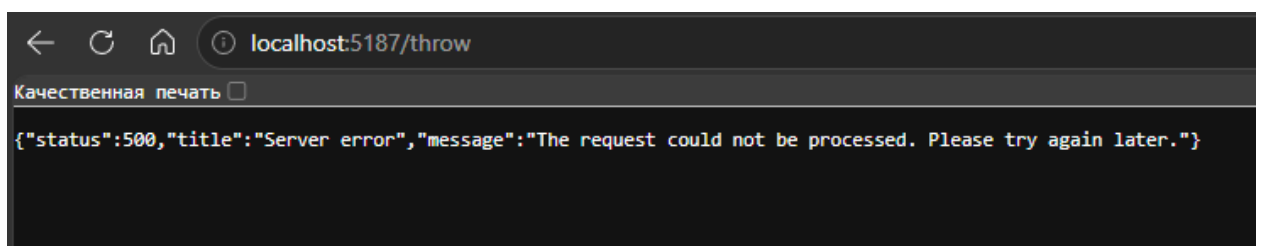
Результат: Браузер показывает подробную страницу с `stack trace`, параметрами запроса и ссылкой на исходный код.



### 3.2. Среда Production

При `ASPNETCORE_ENVIRONMENT=Production` подключается `UseExceptionHandler("/error/exception")`.

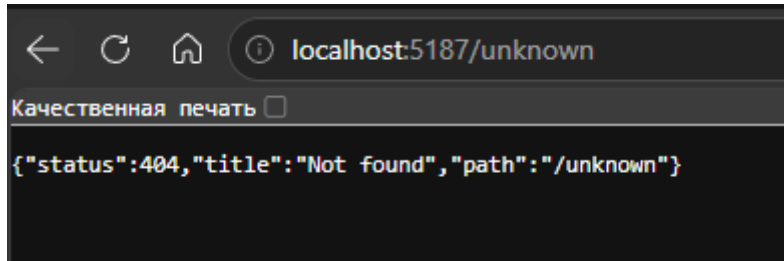
Результат: Исключение перехватывается, логируется, пользователю возвращается безопасный JSON без технических деталей:



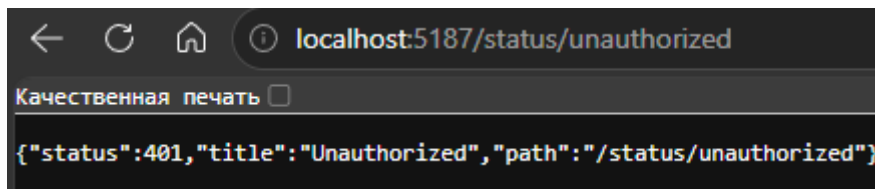
Переключение среды: Выполнено через Properties/launchSettings.json (параметр ASPNETCORE\_ENVIRONMENT: "Production")

#### 4. Обработка HTTP-ошибок

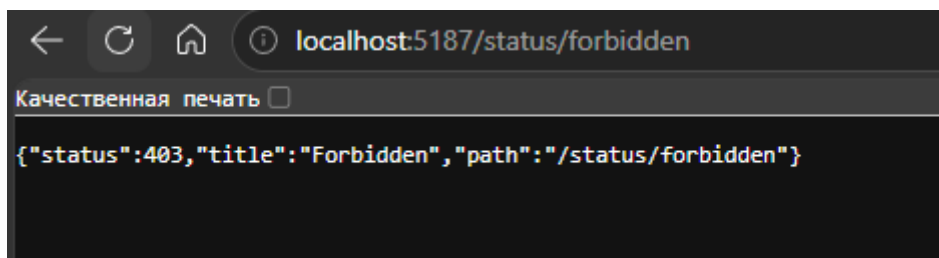
/unknown



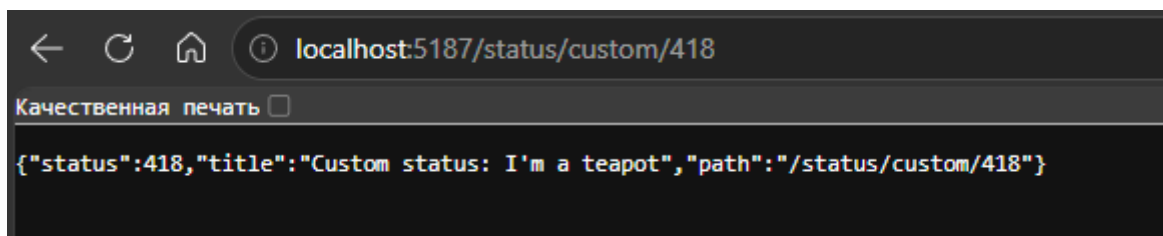
/status/unauthorized



/status/forbidden



/status/custom/418



При отсутствии совпадения маршрутов ASP.NET Core устанавливает статус 404, но не формирует тело ответа. Middleware StatusCodePages intercepts это состояние и вызывает `/error/status/{code}`, который возвращает понятный JSON.

## 5. Порядок Middleware в Pipeline

### Фрагмент Program.cs

```
Program.cs M X
Program.cs
6  var builder = WebApplication.CreateBuilder(args);
7
8  builder.Logging.ClearProviders();
9  builder.Logging.AddConsole();
10 builder.Logging.AddDebug();
11
12 builder.Services.AddSingleton<ITicketRepository, InMemoryTicketRepository>();
13
14 var app = builder.Build();
15
16 if (app.Environment.IsDevelopment())
17 {
18     app.UseDeveloperExceptionPage();
19 }
20 else
21 {
22     app.UseExceptionHandler("/error/exception");
23 }
24
25 app.UseStatusCodePagesWithReExecute("/error/status/{0}");
26 app.UseStaticFiles();
27
28 app.Logger.LogInformation(
29     "HelpDesk.Results started in {Environment} environment",
30     app.Environment.EnvironmentName);
31
32 app.MapGet("/", () => Results.Extensions.Html(homePage));
33
34 app.MapGet("/about/text", () =>
35     Results.Text("HelpDesk.Results returns this response through Results.Text()."));
36
37 app.MapGet("/about/content", () =>
38     Results.Content(
39         "HelpDesk.Results returns this response through Results.Content() with explicit UTF-8 text/plain content.",
40         contentType: "text/plain",
41         contentEncoding: Encoding.UTF8));
42
```

Почему обработчики стоят до конечных точек?

1. UseExceptionHandler должен быть первым, чтобы успеть перехватить любое непойманное исключение, выброшенное в любом последующем middleware или endpoint. Если его разместить после маршрутов, исключение уйдёт клиенту до обработки.
2. UseStatusCodePagesWithReExecute должен стоять до endpoints, потому что он реагирует только на статус-коды 4xx/5xx, у которых ещё нет тела ответа. Если middleware разместить после маршрутов, конечная точка уже завершит обработку, и статусная страница не сможет заменить пустой ответ.

3. UseStaticFiles() размещён после обработчиков ошибок, но до маршрутов API, что позволяет отдавать статику напрямую, не нагружая pipeline маршрутизацией.

## 6. Разница между методами Results API

| Метод                               | Назначение                           | 'Content-Type'                    | Особенности                                                                                               |
|-------------------------------------|--------------------------------------|-----------------------------------|-----------------------------------------------------------------------------------------------------------|
| 'Results.Text (...)'                | Отправка простого текста             | 'text/plain; charset=utf-8'       | Автоматическая кодировка, минимальный контроль над заголовками                                            |
| 'Results.Content(...)'              | Отправка текста с явными параметрами | Задаётся через 'contentType'      | Позволяет явно указать MIME-тип и 'Encoding' (например, 'UTF-8', 'ASCII')                                 |
| 'Results.Json (...)'<br>/ 'Ok(...)' | Сериализация объекта в JSON          | 'application/json; charset=utf-8' | Поддержка 'statusCode', автоматическая сериализация через 'System.Text.Json'                              |
| 'Results.File(...)'                 | Отдача файла/потока/массива байтов   | Задаётся явно                     | Формирует 'Content-Disposition', поддерживает Range-запросы, безопасен при работе с фиксированными путями |

Ключевой принцип: Results API отделяет логику формирования ответа от ручного манипулирования HttpContext.Response, обеспечивая типобезопасность, автоматическую установку заголовков и читаемость кода endpoint'ов.

## 7. Собственная реализация IResult

## HtmlResult.cs

```
C# HtmlResult.cs U X
Results > C# HtmlResult.cs
1 namespace HelpDesk.Results.CustomResults;
2
3 public sealed class HtmlResult : IResult
4 {
5     private readonly string _html;
6     private readonly int _statusCode;
7
8     public HtmlResult(string html, int statusCode = StatusCodes.Status200OK)
9     {
10         _html = html ?? throw new ArgumentNullException(nameof(html));
11         _statusCode = statusCode;
12     }
13
14     public Task ExecuteAsync(HttpContext context)
15     {
16         ArgumentNullException.ThrowIfNull(context);
17
18         context.Response.StatusCode = _statusCode;
19         context.Response.ContentType = "text/html; charset=utf-8";
20
21         return context.Response.WriteAsync(_html);
22     }
23 }
24
```

## HtmlResultExtensions.cs

```
C# HtmlResultExtensions.cs U X
Results > C# HtmlResultExtensions.cs
1 namespace HelpDesk.Results.CustomResults;
2
3 public static class HtmlResultExtensions
4 {
5     public static IResult Html(
6         this IResultExtensions extensions,
7         string html,
8         int statusCode = StatusCodes.Status200OK)
9     {
10         ArgumentNullException.ThrowIfNull(extensions);
11
12         return new HtmlResult(html, statusCode);
13     }
14 }
15
```



Как используется: `Results.Extensions.Html(homePage)` в маршруте `/`. Класс реализует `IResult`, а метод `ExecuteAsync` напрямую записывает HTML в `HttpResponse`, устанавливая корректный `Content-Type`. `Extension`-метод позволяет вызывать результат в едином стиле с остальными методами `Results`.

## Вывод

1. Порядок `middleware` критичен: Обработчики исключений и статус-кодов должны располагаться в начале `pipeline`, иначе они не успеют перехватить ошибки или пустые ответы.
2. Безопасность `Production`-среды: В `Production stack trace` и внутренние детали не должны попадать в ответ. `UseExceptionHandler` + логирование позволяют сохранить диагностику для разработчиков, не раскрывая её пользователям.
3. Статус-код + `Content-Type` + тело ответа: HTTP-ответ всегда состоит из этих трёх компонентов. `Results API` гарантирует их согласованность: например, `Results.Json()` автоматически ставит `application/json`, а `Results.File()` формирует корректный `Content-Disposition` для скачивания.
4. Гибкость `IResult`: Реализация собственного типа позволяет расширять `Minimal API` без потери типобезопасности и единообразия кода. `Extension`-методы делают вызов интуитивным.

Работа продемонстрировала, как `ASP.NET Core` формирует HTTP-ответ на уровне `pipeline`, как правильно изолировать ошибки и как использовать современные `API` (`Results`, `IResult`) для написания чистого, безопасного и поддерживаемого `Minimal API`.